

Interval Selection with Binary Predictions

Christodoulos Karavasilis

University of Toronto

Toronto, Canada

ckar@cs.toronto.edu

ABSTRACT

Following a line of work that takes advantage of vast machine-learned data to enhance online algorithms with (possibly erroneous) information about future inputs, we consider predictions in the context of deterministic algorithms for the problem of selecting a maximum weight independent set of intervals arriving on the real line. We look at two weight functions, unit (constant) weights, and weights proportional to the interval’s length. In the classical online model of irrevocable decisions, no algorithm can achieve constant competitiveness (Bachmann et al. [9] for unit, Lipton and Tomkins [26] for proportional). In this setting, we show that a simple algorithm that is faithful to the predictions is optimal, and achieves an objective value of at least $OPT - \eta$, with η being the total error in the predictions, both for unit, and proportional weights.

When revocable acceptances (a form of *preemption*) are allowed, the optimal deterministic algorithm for unit weights is $2k$ -competitive [12], where k is the number of different interval lengths. We give an algorithm with performance $OPT - \eta$ (and therefore 1-consistent), that is also $(2k + 1)$ -robust. For proportional weights, Garay et al. [19] give an optimal $(2\phi + 1)$ -competitive algorithm, where ϕ is the golden ratio. We present an algorithm with parameter $\lambda > 1$ that is $\frac{3\lambda}{\lambda-1}$ -consistent, and $\frac{4\lambda^2+2\lambda}{\lambda-1}$ -robust. Although these bounds are not tight, we show that for $\lambda > 3.42$ we achieve consistency better than the optimal online guarantee in [19], while maintaining bounded robustness.

We conclude with some experimental results on real-world data that complement our theoretical findings, and show the benefit of prediction algorithms for online interval selection, even in the presence of high error.

KEYWORDS

online algorithms, predictions, interval selection, scheduling

1 INTRODUCTION

We consider the problem of online *interval selection*, or *interval scheduling* on a single machine, where real-length intervals arrive online, and we must output a set of non-conflicting intervals. Each interval is associated with a weight, and the goal is to maximize the sum of weights of the intervals in the solution. This problem is equivalent to finding a maximum weight independent set in interval graphs. We focus on two weight functions, *unit* (or constant) weights, and *proportional* weights, where the weight of an interval is equal to its length. While interval scheduling is often studied under the real-time assumption where intervals arrive in order of non-decreasing starting times, we consider the generalized version of *any-order* arrivals [12]. In the traditional online model of irrevocable decisions, no algorithm (even randomized) can achieve a constant competitive ratio (Bachmann et al. [9] for unit weights,

Lipton and Tomkins [26] for proportional). Because of this, and because some applications permit it, a relaxation of the problem that allows for revocable acceptances has been considered. In this model, every new interval can be accepted by displacing any conflicting intervals in the solution, but every rejection is final. In the area of scheduling, this is sometimes also called *preemption*, although no “restarts” are allowed in our problem. In the offline setting, an optimal solution can be easily found in polynomial time, both for unit, and for proportional weights [23]. The applications of interval scheduling include routing [29], computer wiring [21], project selections during space missions [22], and satellite photography [18]. A more detailed discussion on the applications of interval scheduling can be found in the surveys by Kolen et al. [24] and Kovalyov et al. [25].

Motivated by advancements in machine learning and access to a plethora of data, there has been an effort to equip online algorithms with possibly erroneous predictions about the input instance. Such algorithms are able to achieve much better performance when these predictions are accurate, overcoming some pessimistic bounds of competitive analysis, and helping to bridge the gap between theory and practice. Various classical online problems such as ski rental and non-clairvoyant job scheduling [30], caching [27], facility location [2], metrical task systems [7], and matching [8] have been considered in this model. See Mitzenmacher and Vassilvitskii [28] for a more detailed survey on the topic, and [1] for an online repository of relevant papers. Predictions are also a form of *untrusted advice* (Angelopoulos et al. [3]), a natural extension of the model of online algorithms with advice (Boyar et al. [14]) when the advice is imperfect. Advice research tends to be more information theoretic, focusing on tradeoffs between the number of advice bits and the quality of the solution. Although predictions are often available as offline information, given to the algorithm in advance, we consider a model where a prediction is associated with each input item, and is also given online. This is quite natural and has been considered before for problems such as paging, graph coloring, and packing ([5, 6, 20, 27, 31]). This setting also allows for an oracle to adapt as more of the input is revealed, enabling research where there are different bounds on the quality of later predictions, and allowing one to tailor the predictor algorithm directly [15]. Furthermore, we use binary predictions, which has our model falling in line with work considering limited size, or *succinct* predictions [4, 5, 10].

Related work. Table 1 shows the most relevant existing work in the conventional online setting. In the case of irrevocable decisions, no algorithm (even randomized) can achieve a constant competitive ratio. For the relaxed model of revocable acceptances, we use an asterisk to indicate that the competitive ratio is optimal. In the context of randomized algorithms and revocable acceptances,

Emek et al. [16] give a 6-competitive algorithm for unit weights, while we know of no work improving upon the $(2\phi+1)$ -competitive algorithm.

Table 1: Online results without predictions: n is the size of the input, k is the number of different lengths, Δ is the ratio of the longest to shortest interval.

	Unit	Proportional
Irrevocable (randomized)	$\Omega(n)$ [9]	$\Omega(\log \Delta)$ [26]
Revocable (deterministic)	$2k^*$ [12]	$(2\phi + 1)^*$ [19, 32]

Boyar et al. [13] is the most closely related work to our problem with predictions, and motivated our study. They consider the case of unit weighted intervals on a line graph, and give an optimal deterministic algorithm in the setting of irrevocable decisions with performance $OPT - \eta$ for a different set of predictions and error measure. We extend their work using (possibly adaptive) predictions of limited size, considering an additional weight function of interest, and initiating the study of these problems with revocable decisions.

Structure of the paper. In section 2 we formally define the model, including our predictions and error measure. Section 3 is about the model of irrevocable decisions, whereas in section 4 we allow for revocable acceptances. We conclude with some experiments on real-world data (section 5) that showcase the usefulness of our predictions, and complement our theoretical results.

2 PROBLEM SETTING, DEFINITIONS AND DISCUSSION

In the problem of *interval selection*, an instance consists of a set of intervals arriving on the real line. Each interval is specified by a starting point s_i and an end point f_i , with $s_i < f_i$, and it occupies space $[s_i, f_i]$. The conventional notions of *intersection*, *disjointness*, and *containment* apply. Two intervals can conflict because of a *partial conflict*, or because of *proper inclusion* (figure 1). In the latter case, we say that the smaller (larger) interval is subsumed (subsumes) by the other. Each interval I is also associated with a weight $w(I)$. The goal is to output a set of disjoint intervals of maximum weight. We focus on two types of weight functions, *unit* weights where $w(I) = 1$ for all I , and *proportional* weights where $w(I) = f_i - s_i$. With unit weights, we want to accept as many intervals as possible, whereas with proportional weights, we want to cover as much of the line as possible.

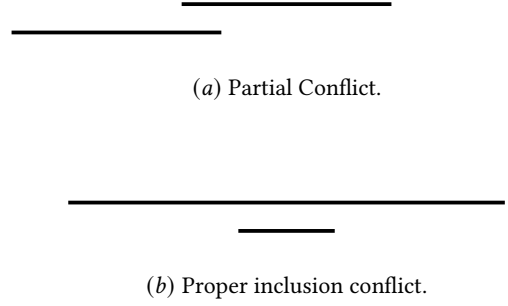


Figure 1: Types of conflicts.

The sequence of the intervals arriving online is chosen by an adversary with full knowledge of the algorithm. In the conventional model of *irrevocable decisions*, each accepted interval is final and will be part of the final solution. A well studied relaxation of the model is allowing for *revocable acceptances*, where any new interval may be accepted by displacing any conflicting intervals in the solution, but every rejection is final. This is sometimes also referred to as *preemption*.

We measure the performance of an online algorithm using *worst case competitive analysis* [11]. Let ALG denote the total weight of the algorithm's solution, and OPT be the total weight of an optimal solution. An algorithm is strictly c -competitive, if for all instances and input permutations, we have that $\frac{OPT}{ALG} \geq c$. We also refer to ALG as the *performance* of the algorithm. Lastly, we will use ALG (respectively OPT) to denote the set of intervals in the algorithm's (optimal) solution. The meaning should always be clear from context.

Predictions. Every interval is associated with a binary prediction that becomes known at the time of the interval's arrival, denoting whether or not that interval is part of some fixed optimal solution. More precisely, $Prd(I) = 1$ means interval I is predicted to be optimal, and $Prd(I) = 0$ means that it is predicted to not be optimal. Let $\mathcal{I}$ denote the set of all intervals in the instance. We define $\vec{p} = (Prd(I_1), Prd(I_2), \dots, Prd(I_{|\mathcal{I}|}))$ to be the binary vector of all the online predictions. Let also $\vec{p}_+$ be the predictions vector when all predictions are accurate, and $\vec{p}_- = \vec{p}_+$.

Error. Every inaccurate prediction introduces an amount of error. Let $\eta(I)$ be the amount of error introduced by interval I . If the prediction of I was accurate, we define $\eta(I) = 0$. If I was wrongly predicted to be non-optimal, let $\eta(I) = w(I)$, and if I was wrongly predicted to be optimal, and C is the set of optimal intervals I conflicts with, let $\eta(I) = \sum_{J \in C} w(J) - w(I)$. Let the total error be $\eta = \sum_{I \in \mathcal{I}} \eta(I)$. One may fix any optimal solution to measure the error against. We use η_{max} to denote the maximum possible error, i.e. $\eta_{max} = \eta$ when $\vec{p} = \vec{p}_-$.

A common approach to evaluate algorithms that use predictions is to focus on an algorithm's *consistency* and *robustness*. We say that an algorithm is γ -consistent if it is γ -competitive, whenever the predictions are accurate, i.e. $\vec{p} = \vec{p}_+$. We say that an algorithm is ζ -robust, if it is ζ -competitive regardless of the accuracy of the

predictions. There is usually a trade-off between consistency and robustness, and the goal is to design algorithms with consistency close to 1, and robustness that is not far worse than the competitiveness of the best predictionless, online algorithm.

Most of our proofs work by using a *charging argument*, where we map the weight of optimal intervals to the weight of intervals taken by the algorithm, and sometimes error. This charging is usually defined in an online manner, and throughout the execution of the algorithm, we use $\Phi(I)$ to refer to the total amount of charge to interval I . In the model of revocable acceptances, we will distinguish between *direct*, and *transfer* charging. Transfer charging (TC) occurs at the moment a new interval is accepted by replacing existing intervals, and refers to the amount of charge it inherits because of this. Direct charge (DC) takes place afterwards, whenever an interval causes optimal intervals to be rejected. We use $TC(I)$ (respectively $DC(I)$) to denote the amount of transfer (direct) charge of an interval, with $\Phi(I) = TC(I) + DC(I)$.

We also use the notion of a *predecessor trace*, which is analogous to Woeginger's [33] *predecessor chain* in the real-time model. If I is an interval in the algorithm's solution, the **predecessor trace** $\mathcal{P}$ of I is the maximal list of intervals $(P_1, P_2, \dots, P_k = I)$, such that P_i was at some point accepted by the algorithm, but was later replaced by P_{i+1} .

3 IRREVOCABLE ACCEPTANCES

In utilizing access to predictions, a natural algorithm to first consider is one that simply *follows the predictions*. We therefore present algorithm 1, which is the main subject of this section.

Algorithm 1 Naive

On the arrival of I :
 $I_s \leftarrow$ Set of intervals currently in the solution conflicting with I
if $Prd(I) = 1$ and $I_s = \emptyset$ **then**
 Take I

Unit weights. We first consider the case of unit weights, and prove the following (positive) result on the performance of algorithm Naive.

THEOREM 3.1. *Algorithm Naive achieves $ALG \geq OPT - \eta$ for interval selection with unit weights.*

PROOF. The proof works by mapping optimal intervals to error, and to intervals taken by the algorithm, given that every missed optimal interval can be associated with at least one unit of error. Let OPT be an optimal solution, and let ALG be the algorithm's solution. For each unit of error, we define an error element h . Let $H = \{h_1, \dots, h_\eta\}$ be the set of error elements. Let $H_I \subseteq H$, be the set of error elements corresponding to the error $\eta(I)$. It holds that $H_I \cap H_J = \emptyset$ for any two distinct intervals I, J , and $\bigcup_I H_I = H$.

We define an injective mapping $F : OPT \rightarrow ALG \cup H$ as follows: Let I_{opt} be an optimal interval. If I_{opt} is taken by the algorithm, it is mapped onto itself. If I_{opt} is not taken by the algorithm, there are two possibilities. The first possibility is that $Prd(I_{opt}) = 1$, but I_{opt} conflicted with another interval I_c taken by the algorithm. For I_c

to have been taken, it means that $Prd(I_c) = 1$, and $|H_{I_c} \cup \{I_c\}| > 0$ because I_c conflicts with I_{opt} . In this case we map I_{opt} to an error element $h_c \in H_{I_c}$, or I_c itself. Even if more optimal intervals were not taken because they conflicted with I_c , there will be enough distinct elements in $H_{I_c} \cup \{I_c\}$ to map them to.

The second possibility is that I_{opt} was not taken, because $Prd(I_{opt}) = 0$. In this case, we have that $|H_{I_{opt}}| = 1$, and we can map I_{opt} to the error element of its own prediction. In conclusion, we have that $ALG + |H| \geq OPT$, and we get the desired bound. $\square$

COROLLARY 3.2. *Algorithm Naive is 1-consistent.*

THEOREM 3.3. *For every deterministic algorithm, there exist a unit weights instance and predictions, such that $ALG = OPT - \eta$.*

PROOF. Let an interval I_{big} arrive first, with $Prd(I_{big}) = 0$. If the algorithm rejects it, no more intervals arrive, and we have $ALG = 0$, $OPT = 1$, $\eta = 1$. If the algorithm takes I_{big} , two non-conflicting intervals arrive next, I_1 and I_2 , both subsumed by I_{big} , with $Prd(I_1) = 0$ and $Prd(I_2) = 1$. In this case, we have $ALG = 1$, $OPT = 2$, $\eta = \eta(I_1) = 1$. In both cases, the equality holds. One can repeat this construction for an asymptotic result. $\square$

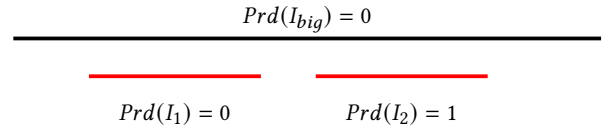


Figure 2: Instance of theorem 3.3.

COROLLARY 3.4. *Algorithm Naive is optimal for unit weights in the model of irrevocable acceptances.*

Boyar et al. [13] were the first to consider the problem of interval selection with unit weights and irrevocable decisions, and they get the same (syntactically) performance, using a different algorithm, and a different set of predictions and error measure. In comparing our result to theirs, we note that our predictions are information theoretically strictly weaker than theirs¹, and can in fact easily be extracted from theirs, allowing our algorithms to operate in their model. Furthermore, their predictions-following algorithm is enhanced with a *greedy* aspect in order to achieve this optimal performance. As we will see in section 5, experimental results on real-world data suggest that for some error ranges, pure *greediness* is arguably a more important attribute than the use of predictions for getting a good solution, and the combination of both in the context of revocable acceptances works best.

We will now show that with **proportional weights**, algorithm Naive achieves the same performance bounds as in the case of unit weights.

THEOREM 3.5. *Algorithm Naive achieves $ALG \geq OPT - \eta$ for interval selection with proportional weights.*

¹Their predictions consist of the entire input instance given in advance.